

Tema 1.2:

Planificación Clásica en

Agentes Inteligentes

Sesión 1: **Representación** de estados y algoritmos fundamentales de búsqueda de planes

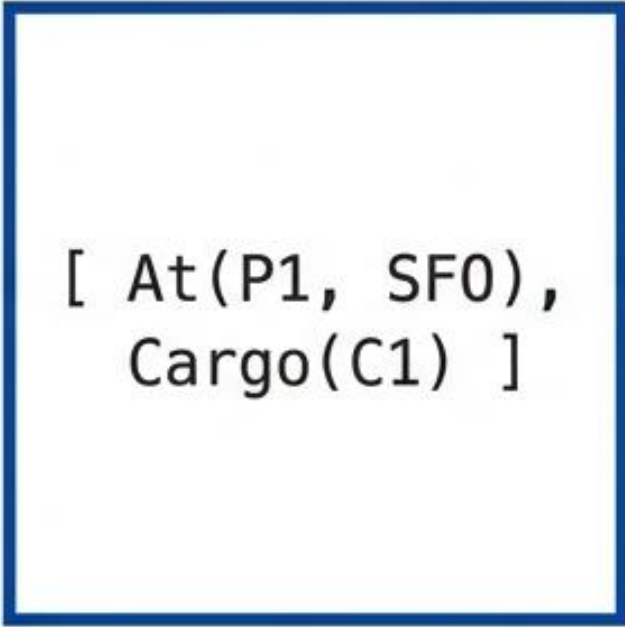
Búsqueda Clásica vs. Planificación Automática



S_0

Representación Atómica

Los estados son nodos indivisibles sin estructura interna. El sistema opera ciegamente guiado solo por funciones heurísticas externas.



[At(P1, SF0),
Cargo(C1)]

Representación Factorizada

Los estados poseen estructura interna definida por variables lógicas. El planificador «lee» esta estructura para deducir la relevancia de una acción sin programación específica.

El Entorno Clásico de Planificación



Entorno Discreto

El tiempo, los estados y las acciones ocurren en pasos bien definidos.



Entorno Determinista

Las acciones tienen exactamente un resultado garantizado.



Entorno Estático

El mundo no cambia mientras el agente delibera.



Entorno Totalmente Observable

El agente conoce todos los aspectos relevantes del estado actual.

Operación en Lazo Abierto (Open-loop)

El plan resultante es una secuencia de acciones fija que no requiere percepción durante la ejecución.

PDDL: Planning Domain Definition Language

Representación de estados mediante conjunciones de fluentes atómicos instanciados (sin variables).

Estado = $\text{At}(\text{Caja1}, \text{Puebla}) \wedge \text{Cargo}(\text{Caja1})$

Semántica de Bases de Datos en PDDL:

Suposición de Mundo Cerrado
(Closed-World Assumption)

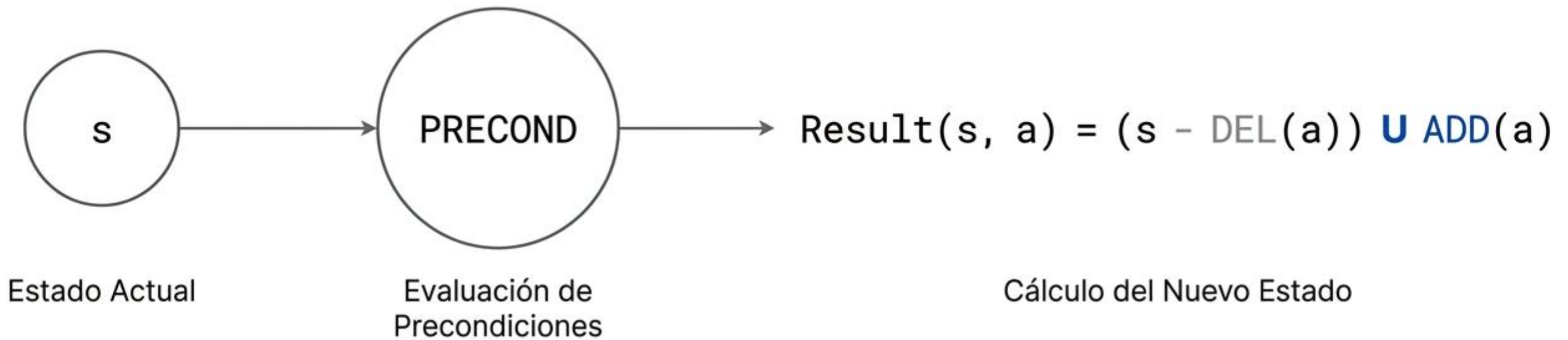
Cualquier fluente no mencionado explícitamente se asume falso. Permite una representación compacta y eficiencia espacial $O(n)$.

Suposición de Nombres Únicos
(Unique Names Assumption)

Las constantes distintas representan objetos físicos distintos en el mundo real (ej. $\text{Caja1} \neq \text{Puebla}$).

Estructura Lógica de los Operadores

Los **Esquemas de Acción (Action Schemas)** definen familias de acciones usando variables libres parametrizadas.



Componentes del Esquema: 1. Parámetros | 2. Precondición (PRECOND) | 3. Efectos (Lista ADD / Lista DEL)

Modelado en Dominios de Referencia



Transporte de Carga Aérea

Enfocado en logística. Consiste en cargar, descargar y volar mercancías entre aeropuertos, evitando inconsistencias de localización espacial.



El Problema de la Llanta de Repuesto

Demuestra la planificación de acciones abstractas que dependen de precondiciones físicas fuertemente secuenciales.

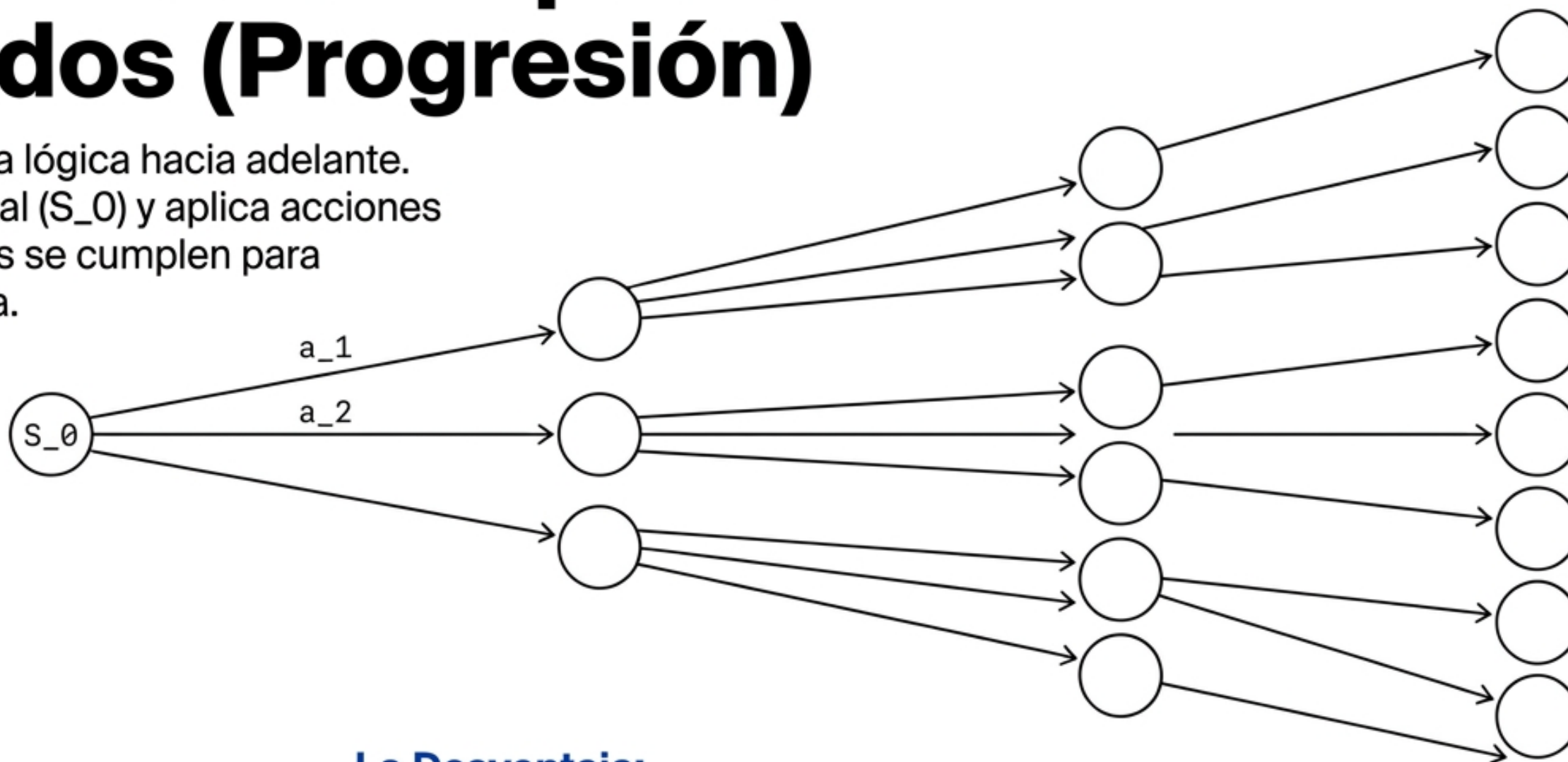


El Mundo de Bloques

Robot manipulando bloques sobre una mesa. Utiliza propiedades lógicas estratégicas como el predicado `Clear(x)` para evitar cuantificadores complejos.

Búsqueda en el Espacio de Estados (Progresión)

Definición: Búsqueda lógica hacia adelante. Parte del Estado Inicial (S_0) y aplica acciones cuyas precondiciones se cumplen para avanzar hacia la meta.



Análisis:

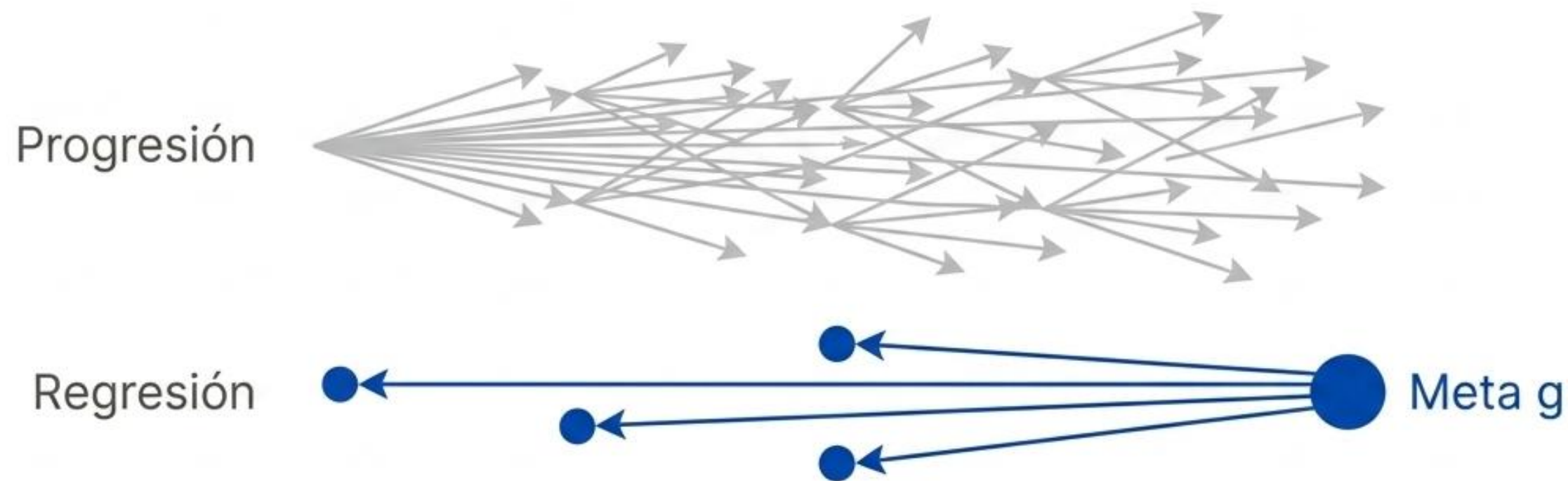
Altamente intuitiva. Simula matemáticamente el avance natural del flujo del tiempo.

La Desventaja:

Produce un inmenso factor de ramificación (explosión combinatoria) si carece de heurísticas informadas, explorando cientos de acciones irrelevantes.

Búsqueda en el Espacio de Fórmulas (Regresión)

Definición: Búsqueda hacia atrás. Parte de la Meta (g) y aplica acciones relevantes de forma inversa. Reduce el factor de ramificación al enfocarse solo en acciones útiles.



Ecuaciones de Regresión de Metas (Nuevo estado meta g'):

$POS(g') = (POS(g) - \text{CobaltBlue}(\text{ADD}(a))) \cup POS(\text{CobaltBlue}(\text{Precond}(a)))$

$NEG(g') = (NEG(g) - \text{CobaltBlue}(\text{DEL}(a))) \cup NEG(\text{CobaltBlue}(\text{Precond}(a)))$

Cierre de la Sesión 1

[Estados Factorizados]

Dotan a la Inteligencia Artificial de una estructura interna comprensible.



[Esquemas PDDL]

Estandarizan el modelado de reglas de transición mediante Precondiciones y Efectos.



[Motores de Búsqueda]

Ejecutan el descubrimiento de planes **combinando estrategias** de Progresión y Regresión.

Actividad de Refuerzo: Modelado en Python

- Misión: Construir la infraestructura básica del planificador clásico.
- Diseño de Clase: Implementar la clase CobaltBlue(Action) para almacenar variables paramétricas, conjunciones de CobaltBlue(PRECOND) y listas CobaltBlue(ADD)/CobaltBlue(DEL).
- Métodos Core: Programar CobaltBlue(verificar_aplicabilidad)(estado, accion) y el procesador lógico CobaltBlue(aplicar_accion)(estado, accion).

```
CobaltBlue(def) aplicar_accion(estado_actual, accion):  
    # Computa: Result(s, a) = (s - DEL(a)) U ADD(a)  
  
    # 1. Ejecutar la Lista de Eliminación  
    estado_sin_del = estado_actual - accion.del_list  
  
    # 2. Ejecutar la Lista de Adición  
    nuevo_estado = estado_sin_del | accion.add_list  
  
    CobaltBlue(return) nuevo_estado
```

Referencias Bibliográficas

Russell, S. J., & Norvig, P. (2020). Inteligencia artificial: Un enfoque moderno (4a ed.). Pearson Education.

Vázquez Mora, P. D. (2026). Syllabus de la Asignatura: Agentes Inteligentes (LIIS1213). Universidad Iberoamericana Puebla.

Google (2026). Gemini Notebook [Asistente de IA generativa].
<https://notebook.google.com>